Hardware Security of DNNs: Side-channel attacks

Sparsh Mittal

Some ideas for launching side-channel attacks

Finding zero vs. non-zero operand

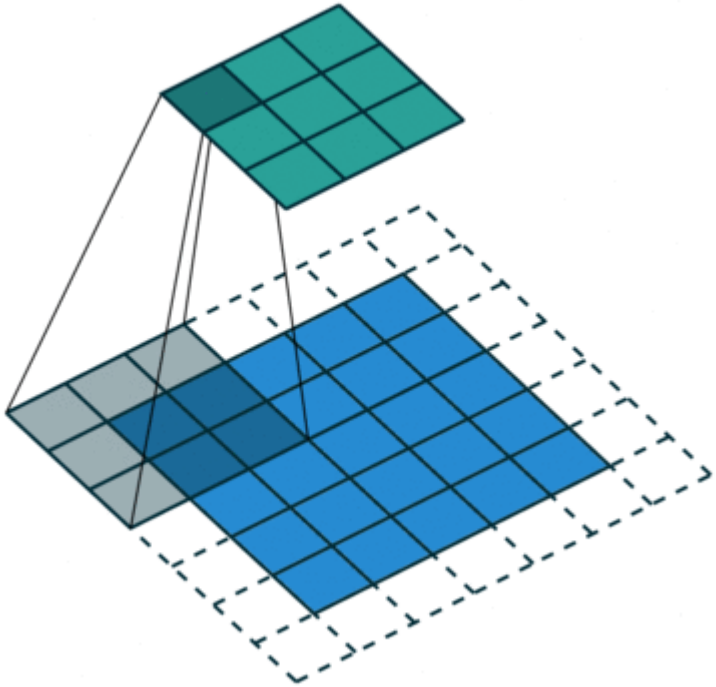
- On microcontrollers, multiplication with zero takes much less time than that with non-zero.
- Assume we track latency of w *I, where w is fixed to 3.14
- We change I to 0.0, 1.0, 2.0, 3.0, 4.0
- The latency of these multiplications are

Input	0	1	2	3	4
Latency	188	300	286	266	284

- Latency when one operand is 0 is significantly shorter than that of the other values.

Correlation for exploiting timing side-channel

- With increasing stride, the execution time is reduced.



Convolution with stride =2

- Runtime of CONV and max-pooling layer increases linearly with the filter size and number of filters.
- These observations can be used to design a substitute DNN of similar accuracy.

Correlation for exploiting timing side-channel

- DNN latency is almost linearly proportional to the number of layers in them

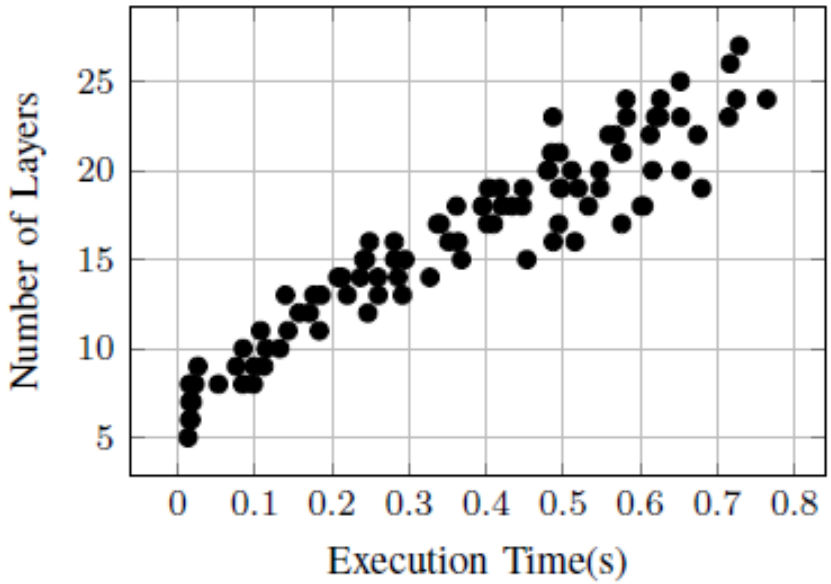


Fig. 4: Scatter Plot of the Adversary's Dataset. Execution time vs Number of layers. Each point in the plot is obtained by varying the parameters of the neural network layers and the number of layers in the neural network. The attacker fits a regressor to predict the depth of the neural network given the execution time.

Correlation for exploiting timing side-channel

- Layer latency is proportional to number of MACs in them

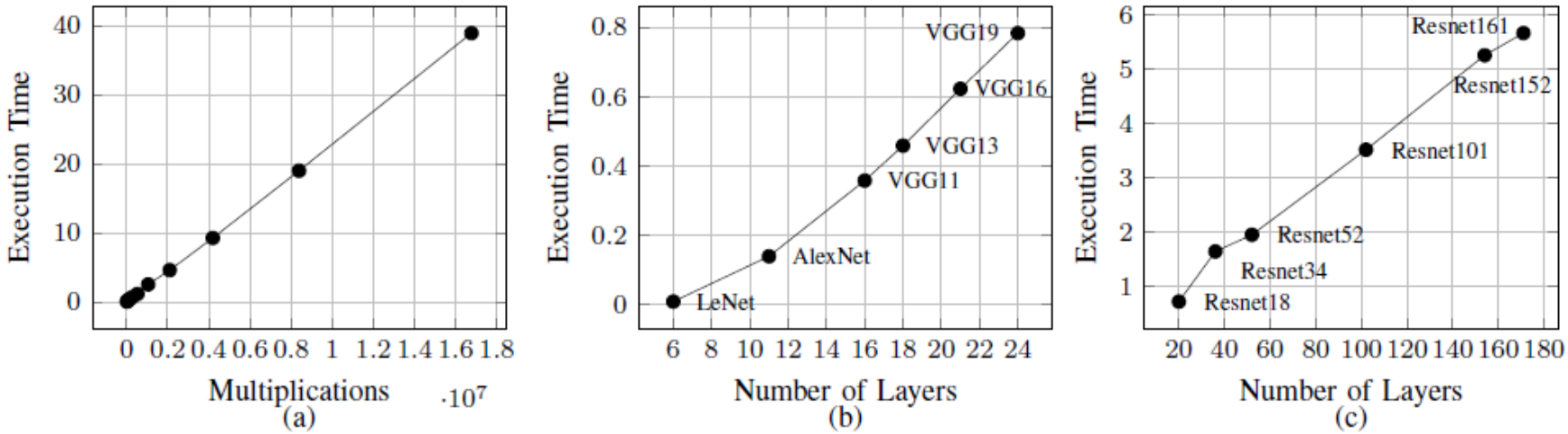
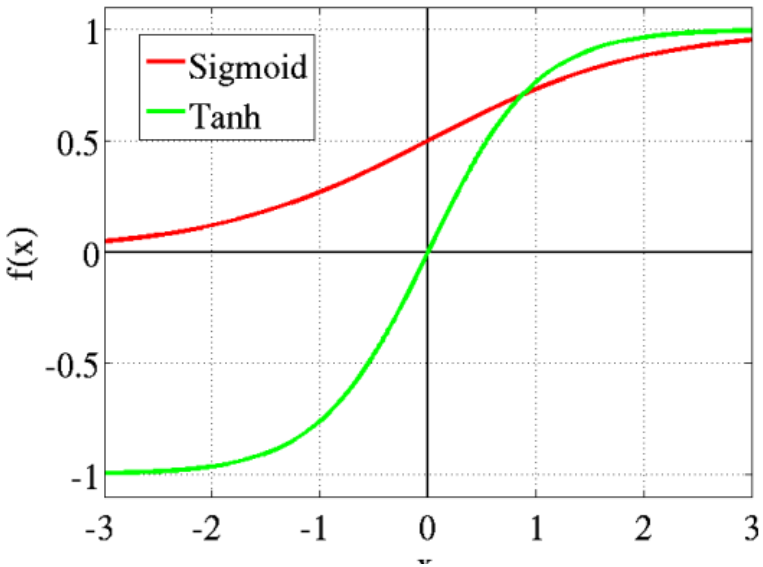


Fig. 3: (a) Fully Connected Layer: Total Execution time varies linearly with the total number of multiplications between the weights of the current layer and the input from the previous layer; (b) Simple Neural Networks Topology: Total Execution Time varies linearly with the number of layers of the neural network architecture; (c) Complex Neural Networks Topology: Total Execution Time varies linearly with the number of layers of the neural network with skip connections between different layers.

Inferring the activation function

- ReLU is simple and does not require support from **libm** library. Tanh/Sigmoid need libm.



$$\text{Sigmoid} = \frac{1}{1+e^{-(x)}}$$

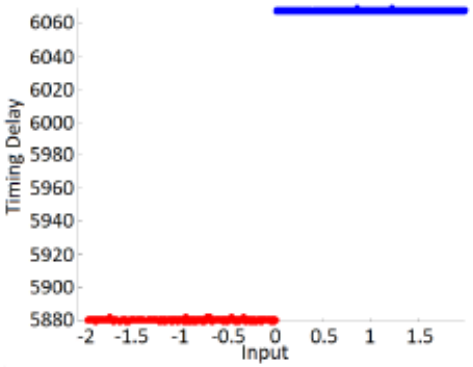
$$tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}.$$

Inferring activation function

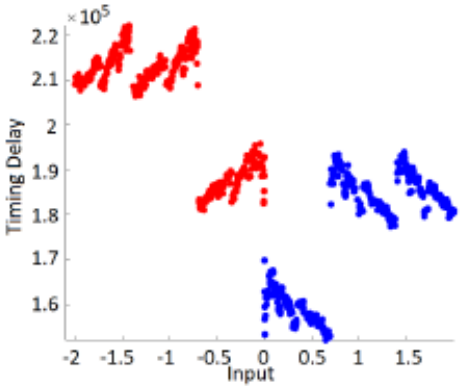
- ReLU has low latency.
- Average and maximum latency are similar for tanh & sigmoid,
- But their minimum values are different: Tanh latency is much lower at input values near zero.
- Tanh latency is symmetric for +ve and -ve values.

Table 1: Minimum, Maximum, and Mean computation time (in *ns*) for different activation functions

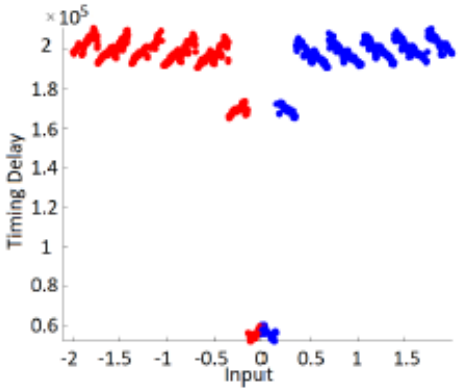
Activation Function	Minimum	Maximum	Mean
ReLU	5 879	6 069	5 975
Sigmoid	152 155	222 102	189 144
Tanh	51 909	210 663	184 864



(a) ReLU



(b) Sigmoid



(c) Tanh

Sparsh Mittal, IIT Roorkee

Timing-correlation

Activation functions (AFs) have conditional branching for implementing division & exponent functions.

Branching operation leads to input-dependent latency differences

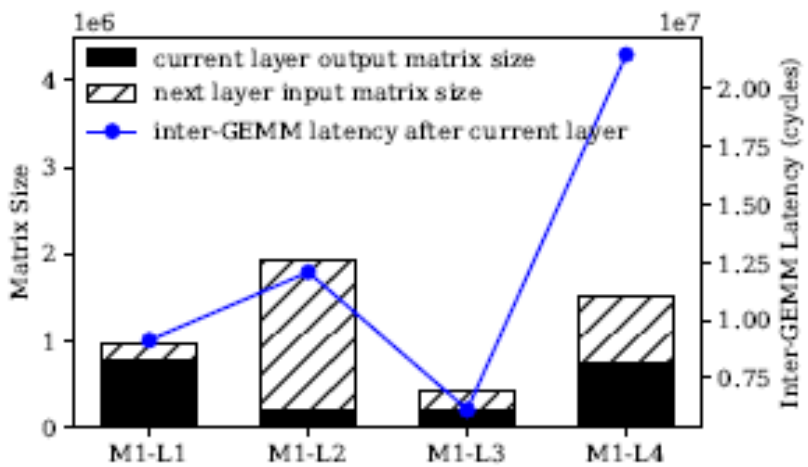
➔By measuring the timing differences, the AF can be ascertained.

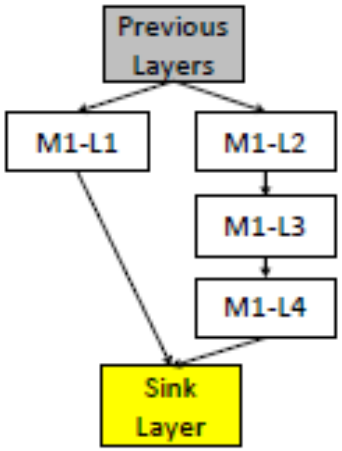
DNN architecture/dimension related

- Filter dimension should be smaller than input feature map width
- Output size of a layer equals input size of next layer.

Distinguishing b/w sequential/non-seq. connection

- Latency between GEMM operations is higher for a non-sequential connection than for a sequential connection
- In below figure, inter-GEMM latency after M1-L4 is much longer than expected. Hence, the layer after M1-L4 is a sink layer.

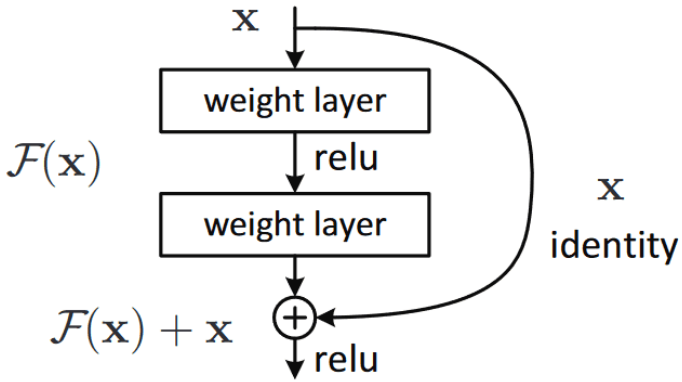




Sparsh Mittal, IIT Roorkee

Inferring skip connections from RAW dependency

- If a particular address was first written, and then read, it leads to RAW dependency.



- When ofmaps from both paths are ready, the memory location is read, and element-wise add is performed.
- Thus, skip connections can be inferred from the RAW dependencies.

RAW= read-after-write, ofmap = output feature map

Exploiting layer characteristics

- FC and CONV layers implement linear transformations of input feature map.
- ReLU applies nonlinear transformation, and BN applies shifting/scaling. Both of them keep the output dimension the same as the input dimension.
- In pooling, output dimension is lower than that of input.
- Addition applies element-wise summation
- Concatenation is performed on sub-input tensors to achieve a single output tensor.

BN = batch normalization

Hardware performance counter-based side-channel

- CONV layers: high data-reuse, low cache misses (if workload fits in the cache)
- FC layer: no reuse (if no batching). ==> High # of cache misses

- Based on this, train an ML classifier to distinguish layer-boundaries

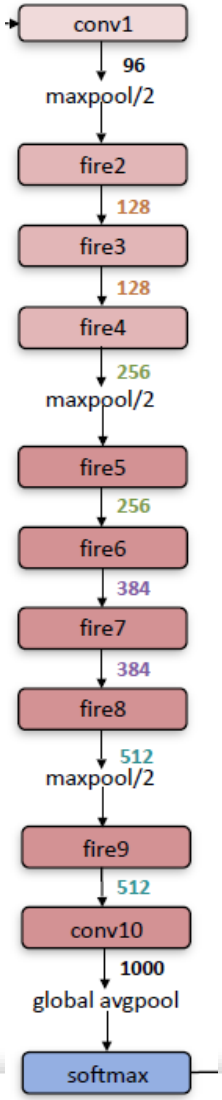
Inferring DNN architecture from building blocks

- Large DNNs are designed by repeatedly connecting the building blocks.
- So, if you can infer the building block, you can infer the DNN.

DNN group	Model Name	Fire module	DWConv	Channel shuffling	Dense block	Inception module	PWConv	Branching	Residual/skip connections	Asymmetric filter decomposition
SqueezeNet	**SqueezeNet-V1.0** [16]	✓	✗	✗	✗	✗	✓	✓	✗	✗
	SqueezeNet-V1.1 [16]	✓	✗	✗	✗	✗	✓	✓	✗	✗
SqueezeNext	1.0-G-SqNxt-23 [9]	✓	✗	✗	✗	✗	✓	✓	✓	✓
	1.0-SqNxt-23 [9]	✓	✗	✗	✗	✗	✓	✓	✓	✓
	1.0-SqNxt-23v5 [9]	✓	✗	✗	✗	✗	✓	✓	✓	✓
	2.0-SqNxt-23 [9]	✓	✗	✗	✗	✗	✓	✓	✓	✓
	2.0-SqNxt-23v5 [9]	✓	✗	✗	✗	✗	✓	✓	✓	✓
MobileNet	**MobileNet-V1** [12]	✗	✓	✗	✗	✗	✓	✗	✗	✗
	MobileNet-V2 [37]	✗	✓	✗	✗	✗	✓	✗	✓	✗
ShuffleNet	**ShuffleNet-V1** [53]	✗	✓	✓	✗	✗	✓	✓	✓	✗
	ShuffleNet-V2 [31]	✗	✓	✓	✗	✗	✓	✓	✓	✗
DenseNet	**DenseNet-121** [15]	✗	✗	✗	✓	✗	✓	✗	✓	✗
InceptionNet	**GoogLeNet** [40]	✗	✗	✗	✗	✓	✓	✓	✗	✗
	Inception-V2 [41]	✗	✗	✗	✗	✓	✓	✓	✗	✓
	SE-BN-Inception [13]	✗	✗	✗	✗	✓	✓	✓	✓	✓

Example of repeated use of building blocks

Fire module in SqueezeNet



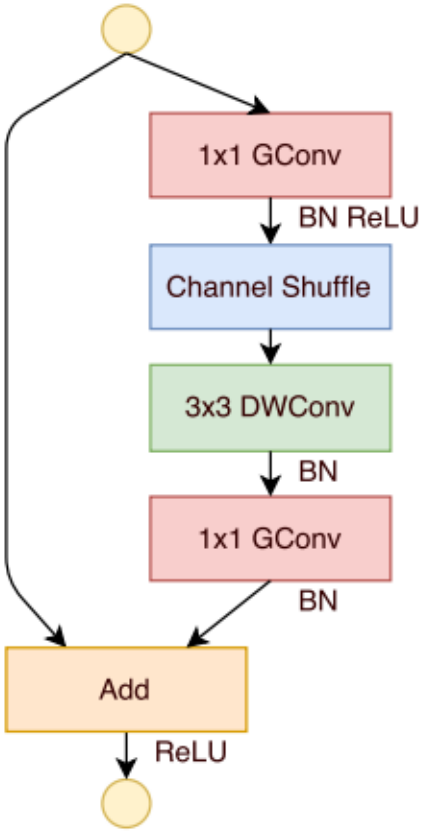
3x3 Depthwise CONV in MobileNet

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
$5\times$ Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

Sparsh Mittal, IIT Roorkee

Example of repeated use of building blocks

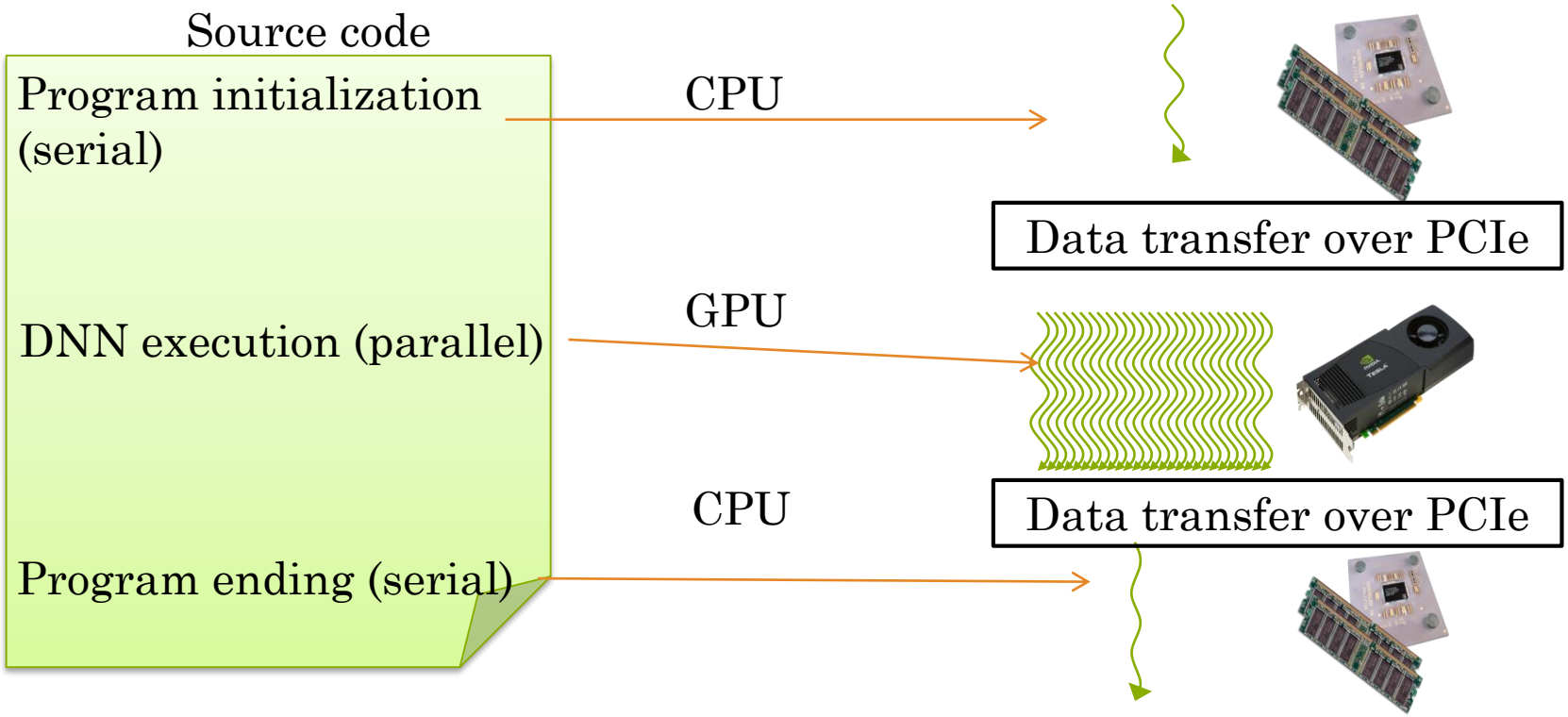


Shufflenet

type
convolution
max pool
convolution
max pool
inception (3a)
inception (3b)
max pool
inception (4a)
inception (4b)
inception (4c)
inception (4d)
inception (4e)
max pool
inception (5a)
inception (5b)
avg pool
dropout (40%)
linear
softmax

InceptionNet

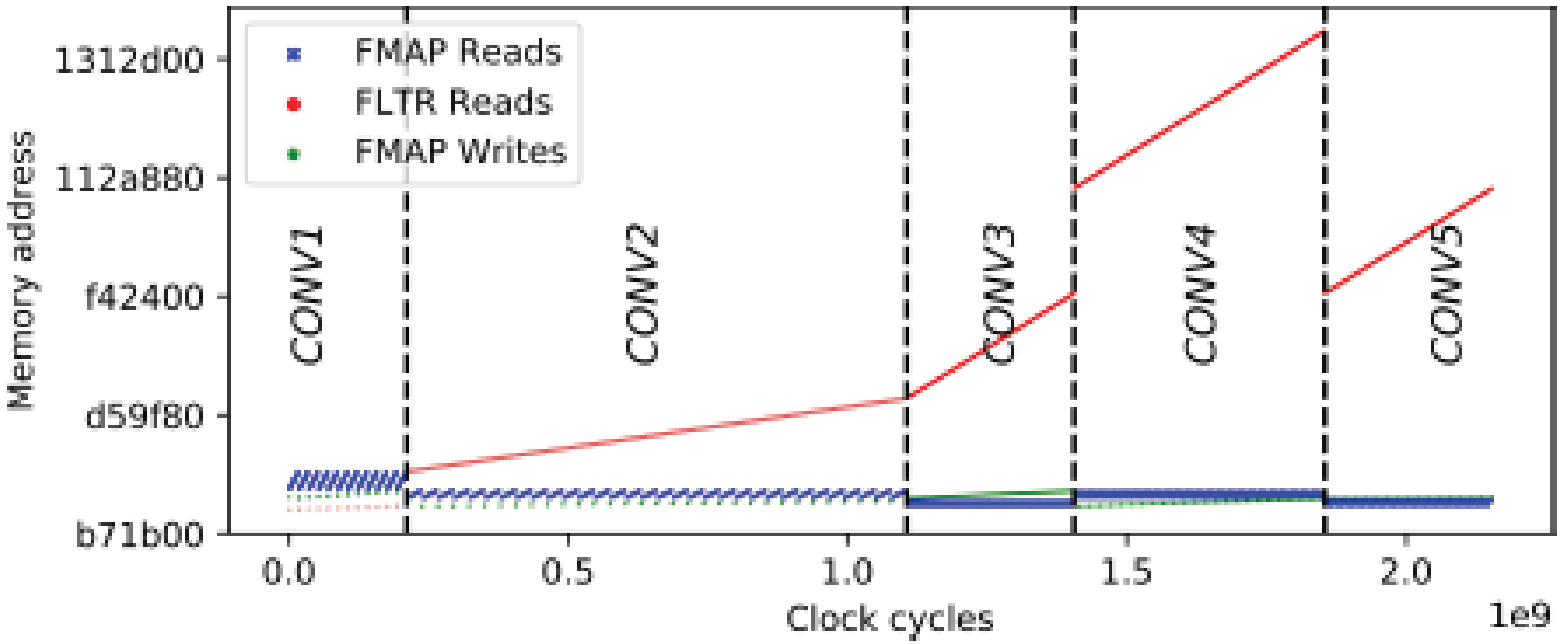
Estimating layer-size



- From memory-copy size on PCIe bus, first layer's input-size and output-size of last layer can be estimated

Ascertaining layer boundaries

- In a single layer, locations storing ofmaps only see (generally one) write-access, whereas previously written locations that are now read store ifmaps.
- Filters and fmaps are stored in contiguous locations as arrays



Ofmap = output feature map, ifmap = input feature map

REFERENCES

- S. Mittal, Himanshi Gupta, Srishti S. “A Survey on Hardware Security of DNN Architectures and Accelerators”, Journal of Systems Architecture, 2021